

Concurrency Design Patterns for Scientific Computing

A Complete Mathematical and Systems Reference

1. The Fundamental Problem

Concurrent programs allow multiple threads of execution to run simultaneously, sharing a common memory space. This is powerful — it lets you overlap expensive computation with I/O, exploit multi-core hardware, and separate concerns into independent processes. But it introduces a class of bugs that are qualitatively different from sequential bugs: they depend on the precise timing of thread execution, which varies between runs, between machines, and even between different load conditions on the same machine. A concurrent bug that occurs once in a thousand runs is often harder to diagnose than a sequential bug that occurs every run.

There are three fundamental hazard classes in concurrent programming.

Data races occur when two threads access the same memory location without synchronization, and at least one access is a write. The result is undefined: you might read a partially-written value, observe an inconsistent combination of old and new values, or encounter platform-specific behavior depending on the processor's memory model. In Python, the Global Interpreter Lock (GIL) prevents true parallel execution of Python bytecode, but it releases between bytecode instructions — which means operations that look atomic in source code, like `dict.update()`, can be interrupted mid-execution.

Deadlocks occur when thread A holds lock X and waits for lock Y, while thread B holds lock Y and waits for lock X. Neither can proceed. This is a cycle in the **lock acquisition graph** — a directed graph where nodes are locks and edges represent “this lock was held when that lock was acquired.” A system is deadlock-free if and only if this graph is acyclic.

Livelocks occur when threads are actively executing — not blocked — but making no overall progress, because each thread's response to another's action undoes the other's progress. Unlike deadlocks, livelocks consume CPU. They are rarer but harder to detect.

2. Lock Hierarchies and Deadlock Freedom

The most reliable technique for preventing deadlocks is the **strict lock hierarchy**: impose a total ordering on all locks in the system, and require that every thread acquires locks in strictly increasing order. If this rule is followed, the lock acquisition graph is acyclically ordered — it can never contain a cycle — and deadlocks are impossible.

Formally, let the locks be $L_1, L_2, \dots, L_k$ with assigned levels $1, 2, \dots, k$. The hierarchy rule says: a thread holding a lock at level i may only acquire locks at level $j > i$. Any attempt to acquire a lock at a lower or equal level while holding a higher-level lock is a **hierarchy violation**.

In a two-level system with application state (Level 1: $\mathcal{L}_{\text{app}}$) and *GP hyperparameters* (Level 2: $\mathcal{L}_{\text{hp}}$, $\mathcal{L}_{\text{data}}$), the rule is:

- A thread may hold $\mathcal{L}_{\text{app}}$ and then acquire $\mathcal{L}_{\text{hp}}$.
- A thread holding $\mathcal{L}_{\text{hp}}$ or $\mathcal{L}_{\text{data}}$ may **never** acquire $\mathcal{L}_{\text{app}}$.

This second rule — that lower-level locks cannot escalate to higher-level locks — is the critical constraint. The background optimizer thread, which holds $\mathcal{L}_{\text{data}}$ briefly during snapshotting and $\mathcal{L}_{\text{hp}}$ briefly during write-back, must never call any application-level method that acquires $\mathcal{L}_{\text{app}}$. This includes logging methods, state update methods, and any other methods that touch application state.

2.1 Runtime Enforcement via a Hierarchical Lock Wrapper

The hierarchy rule is easy to state but easy to violate inadvertently as code evolves. A **hierarchical lock wrapper** enforces the rule at runtime during development, converting silent deadlock risks into immediate exceptions:

```
class HierarchicalLock:
    """
    Wraps a threading.RLock with runtime hierarchy checking.

    Each thread maintains a thread-local set of held levels.
    Attempting to acquire a lock at a level  $\leq$  any currently held
    level raises RuntimeError immediately rather than silently deadlocking.

    This should be used during development and testing. In production,
    you can replace with a plain threading.RLock for zero overhead,
    having gained confidence that hierarchy is maintained.
```

```

"""
_held_levels = threading.local()  # Thread-local: each thread tracks its own

def __init__(self, level: int, name: str):
    self.level = level
    self.name = name
    self._lock = threading.RLock()

def acquire(self, blocking=True, timeout=-1):
    held = getattr(self._held_levels, 'levels', set())

    # Check for violations: are we trying to go "upward" to a lower level?
    violations = [l for l in held if l < self.level]
    if violations:
        raise RuntimeError(
            f"Lock hierarchy violation: attempting to acquire '{self.name}' "
            f"(level {self.level}) while holding locks at levels {violations}"
            f"This is a potential deadlock."
        )

    result = self._lock.acquire(blocking=blocking, timeout=timeout)
    if result:
        if not hasattr(self._held_levels, 'levels'):
            self._held_levels.levels = set()
        self._held_levels.levels.add(self.level)
    return result

def release(self):
    self._lock.release()
    if hasattr(self._held_levels, 'levels'):
        self._held_levels.levels.discard(self.level)

def __enter__(self):
    self.acquire()
    return self

def __exit__(self, *args):
    self.release()

```

The key insight is that hierarchy violations are **structural** — they reflect an architectural problem with how locks are used — while deadlocks are **temporal** — they require a specific unfortunate interleaving of execution. By converting structural violations into immediate exceptions, you catch the problem deterministically during testing rather than waiting for the rare timing condition that produces a deadlock in production.

3. The Snapshot Pattern and Immutability

The snapshot pattern is the primary mechanism for minimizing lock hold time in systems with expensive computations. The principle is simple: acquire the lock, copy all needed state into a local (immutable) structure, release the lock, then perform all expensive computation on the copy.

```
def get_snapshot(state) -> dict:
    """
    Acquires the lock only long enough to copy state into a plain dict.
    All downstream computation works on this immutable snapshot, holding no locks

    This pattern is the key to preventing lock contention: the lock is held
    for microseconds (the copy), not for the duration of the computation
    (potentially milliseconds to seconds).
    """
    with state.lock: # Lock held only during this block
        return {
            "rssi": state.target.kalman.x,
            "clients": list(state.target.clients.keys()),
            "logs": list(state.logs),
            # ... copy all needed state ...
        }

# --- All computation below holds no locks ---
snapshot = get_snapshot(state)
result = expensive_computation(snapshot) # Can take arbitrarily long
```

The theoretical basis for this pattern is the distinction between **safe publication** and **safe access**. An object is safely published if it is made visible to other threads in a consistent state. An object is safely accessed if reads and writes are synchronized. Immutable objects require only safe publication — once created and made visible, they can be read concurrently without any synchronization, because there is nothing to race against.

4. The Producer-Consumer Pattern and Message Passing

The producer-consumer pattern is a foundational concurrency design that eliminates shared mutable state between two components by interposing a queue. The **producer** thread generates results and deposits them in the queue. The **consumer** thread retrieves results from the queue and processes them. The queue handles all synchronization internally.

This pattern is an instance of the **message-passing paradigm** — one of the two major approaches to concurrent programming (the other being shared memory with locks). The theoretical foundation is Tony Hoare’s **Communicating Sequential Processes** (CSP, 1978) and the closely related **Actor Model** (Hewitt et al., 1973).

The key insight of message passing is that each piece of mutable state is **owned** by exactly one thread, which is the only thread allowed to read or modify it. Other threads communicate with it by sending immutable messages through channels. This eliminates all data races by design — there is no shared mutable state to race on.

In the GP hyperparameter optimization context, the producer-consumer pattern is used as follows. The background optimizer **owns** the optimization computation — it is the only thread that runs L-BFGS-B. The worker thread **owns** the decision-making loop. They communicate via a `SimpleQueue`:

```
# Background optimizer (producer): deposits results, never touches AppState
result = run_optimization(xs_snapshot, ys_snapshot)
if result is not None:
    hp_queue.put(result) # Non-blocking, lock-free

# Worker thread (consumer): picks up results at a safe moment
def apply_pending_updates(hp_queue, mogp):
    """
    Called at the start of each worker iteration, outside AppState.lock.
    Drains any pending hyperparameter updates.
    """
    try:
        while True:
            result = hp_queue.get_nowait() # Non-blocking
            with mogp._hp_lock:           # Level-2 lock only
                mogp.length_scale = result['ls']
                mogp.noise = result['noise']
                mogp._L_B = result['_L_B'] # Atomic joint update
    except queue.Empty:
        pass
```

The atomic joint update of `length_scale`, `noise`, and `_L_B` under a single lock acquisition is essential. These three parameters were optimized jointly — they are only consistent as a triple. If you updated them in separate lock acquisitions, the worker thread could read a new `length_scale` with the old `_L_B` between those acquisitions, computing a posterior based on an incoherent model.

5. The Event-Signal Pattern for Thread Lifecycle Management

Threads need mechanisms to communicate lifecycle events: "new data is available", "please shut down", "a fatal error has occurred." The `threading.Event` class provides a simple and efficient mechanism: `event.set()` signals the event, `event.wait(timeout)` blocks until the event is signaled or the timeout expires, and `event.clear()` resets it to the unsignaled state.

The key design choices when using events are the **granularity of signaling** (signal on every update vs. signal periodically) and the **semantics of clearing** (who clears the event, and when).

For hyperparameter optimization triggered by new data, the appropriate pattern is:

```
# In the data-writing thread (signal after releasing lock):
with data_lock:
    data.append(new_observation)
# Signal AFTER releasing the lock so the optimizer can immediately acquire it
new_data_event.set()

# In the optimizer thread:
while not stop_event.is_set():
    triggered = new_data_event.wait(timeout=5.0) # Timeout prevents eternal sleep

    if stop_event.is_set():
        break

    if not triggered:
        continue # Timeout — no new data, loop again

# Clear BEFORE snapshotting. If new data arrives while we're optimizing,
# the event will be re-set and we'll catch it on the next iteration.
# If we cleared AFTER snapshotting, we might miss data that arrived
# during the optimization run.
new_data_event.clear()

with data_lock:
    snapshot = copy_data()

result = run_optimization(snapshot) # Lock-free, can take arbitrarily long
```

The ordering of `clear()` before the snapshot is subtle but important. If you clear after snapshotting, new data that arrives during optimization will be missed — the event is cleared after you've already committed to your snapshot. If you clear before snapshotting, new data that arrives during optimization will set the event again, ensuring you re-run

optimization on the next loop iteration and pick up the new data.

6. Concurrent Phase Annotation as a Design Tool

A powerful technique for making lock discipline visible and reviewable is to **annotate the concurrency phase** of each section of code. The worker thread's main loop naturally divides into phases with different lock states:

```
while not shutdown.is_set():

    # — PHASE 1: READ STATE —————
    # Acquires and releases AppState.lock internally via get_snapshot().
    # After this line, no locks are held.
    snapshot = get_snapshot(state)

    # — PHASE 2: EXPENSIVE COMPUTATION —————
    # Acquires _data_lock and _hp_lock internally (Level 2 only).
    # Never acquires AppState.lock. May take milliseconds.
    arm, intensity = mogp.joint_thompson_sample(grid, current_time)

    # — PHASE 3: I/O —————
    # No locks held. Pure I/O — send packet, wait for response.
    execute_arm, intensity)
    time.sleep(wait_time)

    # — PHASE 4: OBSERVE AND UPDATE MODELS —————
    # Acquires Level-2 locks internally via mogp.observe().
    # Also calls state.log() which briefly acquires AppState.lock.
    # Safe because we hold no locks when entering this phase.
    reward = compute_reward(get_snapshot(state))
    mogp.observe(arm, intensity, current_time, reward)

    # — PHASE 5: WRITE STATS —————
    # Brief AppState.lock acquisition — minimal work inside the lock.
    with state.lock:
        state.stats.update({"arm": arm, "rew": reward})
```

These annotations serve two purposes. During development, they make the lock protocol explicit and reviewable — a reviewer can immediately see what invariants hold at each point and check that no phase violates the hierarchy. After deployment, they serve as documentation that describes the concurrent behavior of the code in terms that are meaningful to future maintainers.

The annotations transform lock discipline from an implicit convention (which is easy to violate accidentally) into an explicit specification (which can be reviewed, tested, and enforced).

7. Testing Concurrent Systems

Testing concurrent code is genuinely hard, because the bugs are timing-dependent. Three complementary approaches provide increasing coverage.

Stress testing runs the concurrent system under high load for extended periods, maximizing the chance that unfortunate thread interleavings occur. While not systematic, stress testing often reveals bugs that would take weeks to surface in normal use.

Runtime detection uses instrumented lock wrappers (like `HierarchicalLock` above) to detect violations immediately when they occur, converting rare timing-dependent deadlocks into deterministic exceptions.

Model checking uses tools like Spin or TLA+ to formally specify the concurrent protocol and exhaustively explore all possible thread interleavings, providing a mathematical proof of correctness for the specified protocol. This is expensive to apply but provides the strongest guarantees — it is used in safety-critical systems like aerospace software and database transaction managers.

For scientific computing applications, runtime detection combined with stress testing typically provides sufficient confidence. The key invariant to verify is that the lock acquisition graph is acyclic — and the `HierarchicalLock` wrapper detects cycles in this graph at the moment they would form.

See also: The Two-Timescale Bayesian Update; Gaussian Process Regression